1 Security today

This chapter covers

- What Spring Security is and what you can solve by using it
- What security is for a software application
- Why software security is essential and why you should care

Developers have become increasingly aware of the need for secure software, and they have been taking responsibility for security from the beginning of software development. Generally, as developers, we begin by learning that the purpose of an application is to solve business problems. This purpose refers to something where data could be processed somehow, persisted, and eventually displayed to the user as specified by requirements. This overview of software development, which is somehow imposed from the early stages of learning these techniques, has the unfortunate disadvantage of hiding practices that are also part of the process. While the application works correctly from the user's perspective, and it eventually does what the user expects in terms of functionality, there are many aspects hidden in the final result.

Nonfunctional software qualities such as performance, scalability, availability, and security, as well as others, can have an effect over time, from the short to the long term. If not taken into consideration early on, these qualities can dramatically affect the profitability of the application owners (figure 1.1). Moreover, the neglect of these considerations can also trigger failures in other systems (for example, by the unwilling participation in a distributed denial of service [DDoS] attack). The hidden aspects of nonfunctional requirements (the fact that it's much more challenging to see if something's missing or incomplete) make these even more dangerous.

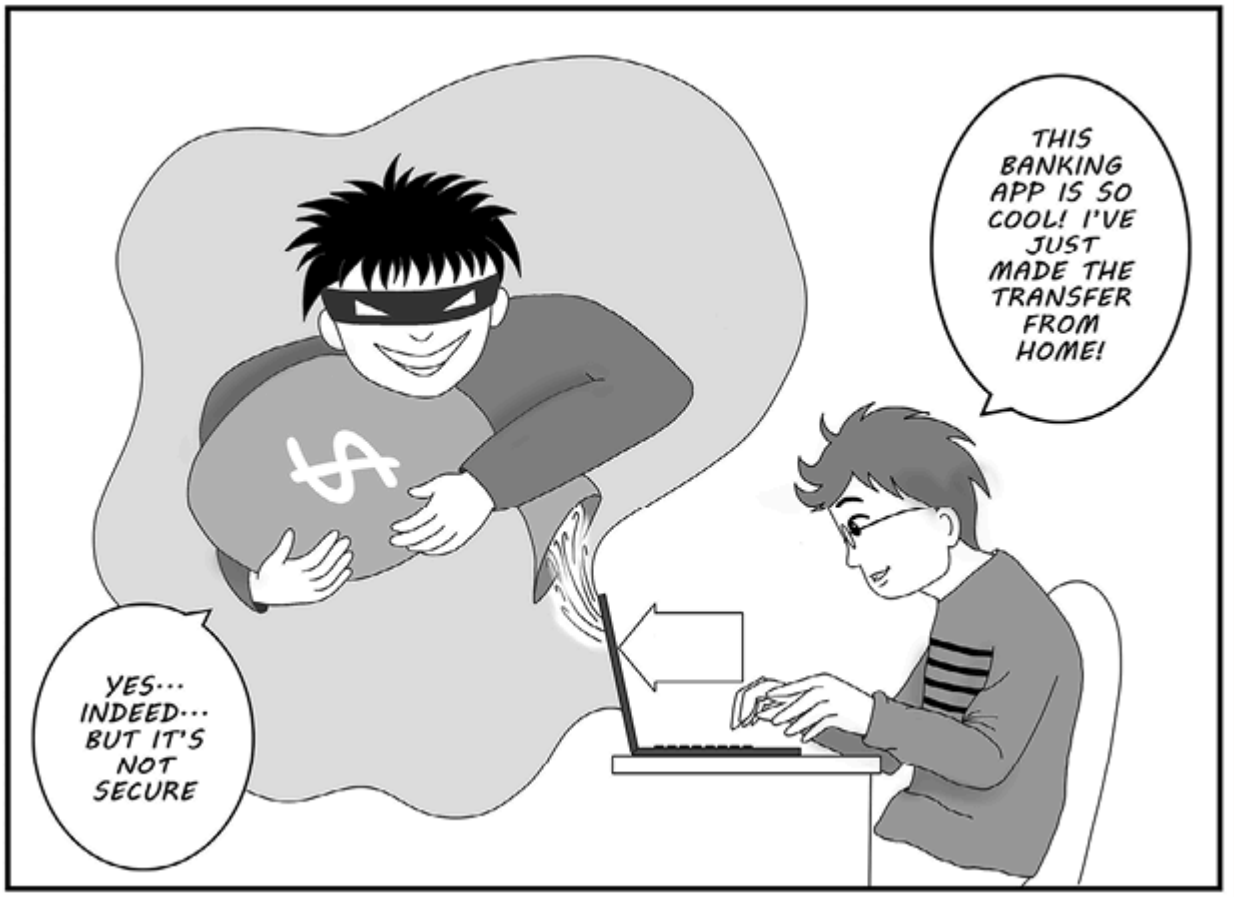


Figure 1.1 Users typically focus on what a system should do—its functional aspects. Occasionally, they may consider system performance, a nonfunctional attribute, but it's rare for them to pay attention to security measures. Requirements that don't pertain directly to functionality often go unnoticed compared to those that do.

There are multiple nonfunctional aspects to consider when working on a software system. In practice, all of these are important and need to be treated responsibly in the process of software development. This book focuses on one of these: security. You'll learn how to protect your application, step by step, using Spring Security.

This chapter will show you the big picture of security-related concepts. Throughout the book, we will work on practical examples, and where appropriate, I'll refer back to the description I give in this chapter. Where applicable, I'll also provide you with more details. Here and there, you'll find references to other materials (books, articles, and documentation) on specific subjects useful for further reading.

1.1 Discovering Spring Security

This section discusses the relationship between Spring Security and Spring. First, it is important to understand the link between the two before starting to use them. If we check the official website (**https://spring.io/projects/spring-security**), we can see that Spring Security is described as a powerful and highly customizable framework for authentication and access control. I would simply say it is a framework that greatly simplifies applying (or baking in) security for Spring applications.

Spring Security is the primary choice for implementing application-level security in Spring applications. Generally, its purpose is to offer you a highly customizable way of implementing authentication, authorization, and protection against common attacks. Spring Security is open source software released under the Apache 2.0 license. You can access its source code on GitHub at **http://mng.bz/vPmJ**. I highly recommend that you contribute to the project as well.

NOTE You can use Spring Security for both standard web servlets and reactive applications, as well as non-web apps. In this book, we'll use Spring Security with the latest Java long-term supported, Spring, and Spring Boot versions (Java 21, Spring 6, and Spring Boot 3). However, all the book's examples also work with Java 17, the previous long-term supported version.

I can guess that if you opened this book, then you work on Spring applications, and you are interested in securing those. Spring Security is most likely the best choice for you. It's the de facto solution for implementing application-level security for Spring applications. Spring Security, however, doesn't automatically secure your application. It's not some kind of magic panacea that guarantees a vulnerability-free app. Developers need to understand how to configure and customize Spring Security around the needs of their applications. How to do this depends on many factors, from the functional requirements to the architecture.

Technically, applying security with Spring Security in Spring applications is simple. You've already implemented Spring applications, so you know that the framework's philosophy starts with the management of the Spring context. You define beans in the Spring context to allow the framework to manage these based on the configurations you specify.

You use annotations to tell Spring what to do: expose endpoints, wrap methods in transactions, intercept methods in aspects, and so on. The same is true with Spring Security configurations, which is where Spring Security comes into play. You want to use annotations, beans, and in general, a Spring-fashioned configuration style comfortably when defining your application-level security. In a Spring application, the behavior that you need to protect is defined by methods.

To think about application-level security, you can consider your home and the way you allow access to it. Do you place the key under the entrance rug? Do you even have a key for your front door? The same concept applies to applications, and Spring Security helps you develop this functionality. It's a puzzle that offers plenty of choices for building the

exact image that describes your system. You can choose to leave your house completely unsecured, or you can decide not to allow everyone to enter your home.

The way you configure security can be straightforward like hiding your key under the rug, or it can be more complicated like choosing a variety of alarm systems, video cameras, and locks. In your applications, you have the same options, but as in real life, the more complexity you add, the more expensive it gets. In an application, this cost refers to the way security affects maintainability and performance.

But how do you use Spring Security with Spring applications? Generally, at the application level, one of the most frequently encountered use cases is when you're deciding whether someone is allowed to perform an action or use some piece of data. Based on configurations, you write Spring Security components that intercept the requests and that ensure whoever makes the requests has permission to access protected resources. The developer configures components to do precisely what's desired. If you mount an alarm system, it's you who should make sure it's set up for the windows as well as for the doors. If you forget to set it up for the windows, it's not the fault of the alarm system that it doesn't trigger when someone forces a window.

Other responsibilities of Spring Security components relate to data storage as well as data transit between different parts of the systems. By intercepting calls to these different parts, the components can act on the data. For example, when data is stored, these components can apply encryption or hashing algorithms. The data encodings keep the data accessible only to privileged entities. In a Spring application, the developer has to add and configure a component to do this part of the job wherever it's needed. Spring Security provides us with a contract through which we know what the framework requires to be implemented, and we write the implementation according to the application design. We can say the same thing about transiting data.

In real-world implementations, you'll find cases in which two communicating components don't trust each other. How can the first know that the second one sent a specific message, and it wasn't sent by someone else? Imagine you have a phone call with somebody to whom you must give private information. How do you make sure that a valid individual with the right to get that data is indeed on the other end, and not somebody else? The same situation is relevant to your application. Spring Security provides components that allow you to solve these problems in several ways, but you must know which part to configure and then set it up in your system. In this way, Spring Security intercepts messages and makes sure to validate communication before the application uses any kind of data sent or received.

Like any framework, one of the primary purposes of Spring is to allow you to write less code to implement the desired functionality. This is also what Spring Security does. It completes Spring as a framework by helping you write less code to perform one of the most critical aspects of an application—security. Spring Security provides predefined functionality to help you avoid writing boilerplate code or repeatedly writing the same logic from app to app. However, it also allows you to configure any of its components, thus providing great flexibility. To briefly recap this discussion:

- You use Spring Security to bake application-level security into your applications in the Spring way. By this, I mean you use annotations, beans, the Spring Expression Language (SpEL), and so on.
- Spring Security is a framework that lets you build application-level security. However, it is up to you, the developer, to understand and use Spring Security properly. Spring Security by itself does not secure an application or sensitive data at rest or in flight.
- This book provides you with the information you need to effectively use Spring Security.

Alternatives to Spring Security

This book is about Spring Security, but as with any solution, I always prefer to have a broad overview. Never forget to learn the alternatives that you have for any option. One of the things I've learned over time is that there's no general right or wrong. The saying "Everything is relative" also applies here!

You won't find a lot of alternatives to Spring Security when it comes to securing a Spring application. One alternative you could consider is Apache Shiro (**https://shiro.apache.org**). It offers flexibility in configuration and is easy to integrate with Spring and Spring Boot applications. Apache Shiro sometimes makes a good alternative to the Spring Security approach.

If you've already worked with Spring Security, you'll find using Apache Shiro easy and comfortable to learn. It offers its own annotations and design for web applications based on HTTP filters, which greatly simplifies working with web applications. Also, you can secure more than just web applications with Shiro, from smaller command-line and mobile applications to large-scale enterprise applications. And although simple, it's powerful enough to be used for a wide range of things—from authentication and authorization to cryptography and session management.

However, Apache Shiro could be too light for the needs of your application. Spring Security is not just a hammer, but an entire set of tools. It offers a larger scale of possibilities and is designed specifically for Spring applications. Moreover, it benefits from a larger community of active developers, and it is continuously enhanced.

1.2 What is software security?

Software systems manage large amounts of data, of which a significant part can be considered sensitive, especially in some parts of the world, for example, given the European General Data Protection Regulations (GDPR) requirements. Any information that you, as a user, consider private is sensitive for your software application. Sensitive data can include harmless information such as a phone number, email address, or identification number, although we generally think more about data that is riskier to lose, like your credit card details. The application should ensure that there's no chance for that information to be accessed, changed, or intercepted. No parties other than the users for whom this data is intended should be able to interact in any way with it. Broadly expressed, this is the meaning of security.

NOTE GDPR created a lot of buzz globally after its introduction in 2018. It generally represents a set of European laws that refers to data protection and gives people more control over their private data. GDPR applies to the owners of systems that have users in Europe. The owners of such applications risk significant penalties if they don't respect the regulations imposed.

We apply security in layers, with each layer requiring a different approach. Compare these layers to a protected castle (figure 1.2). A hacker needs to bypass several obstacles to obtain the resources managed by the app. The better you secure each layer, the lower the chance that an individual with bad intentions manages to access data or perform unauthorized operations.

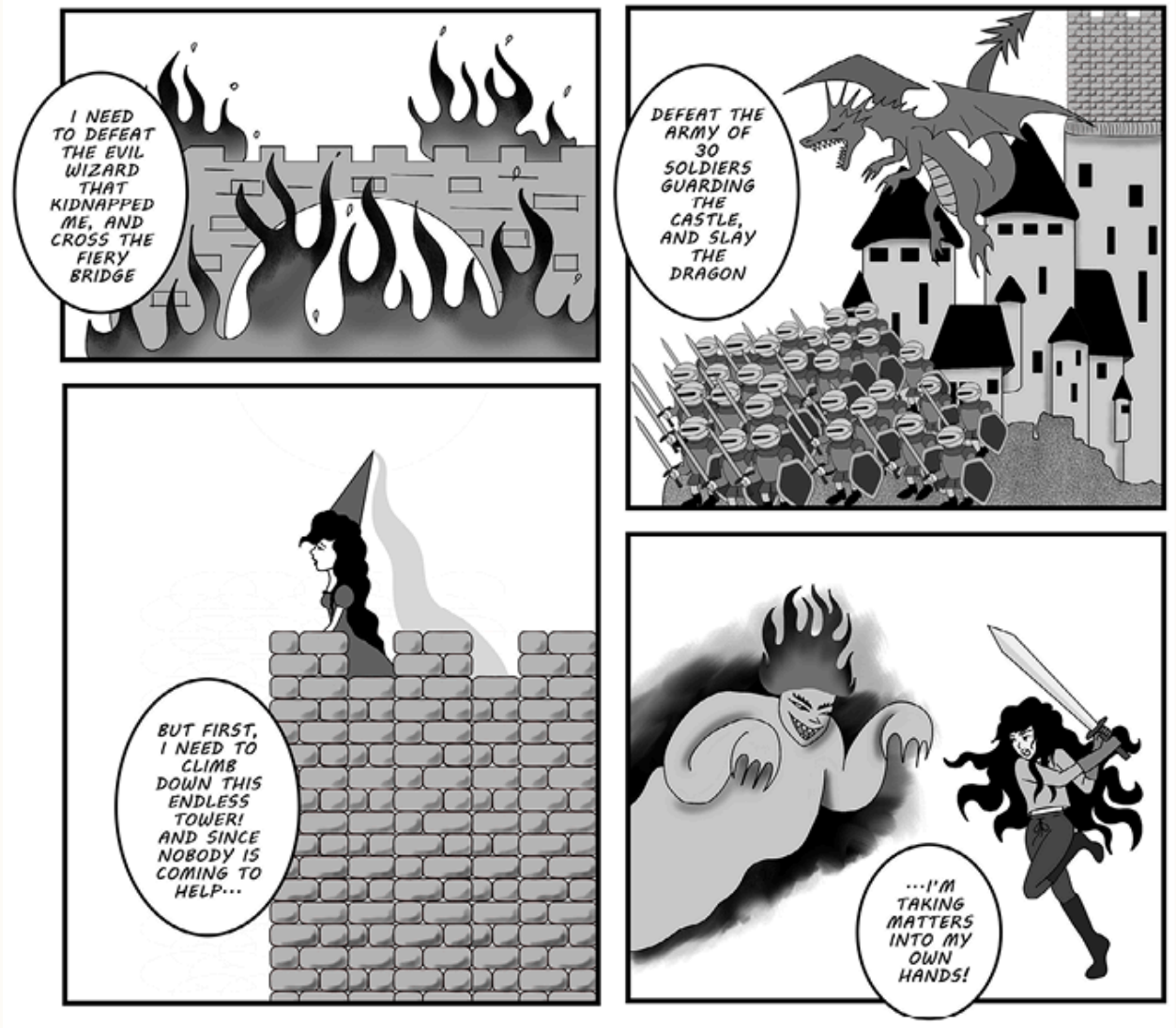


Figure 1.2 The Dark Wizard (a hacker) must bypass multiple obstacles (security layers) to steal the Magic Sword (user resources) from the Princess (your application).

Security is a complex subject. In the case of a software system, security doesn't exist only at the application level. For example, for networking, there are problems to be taken into consideration and specific practices to be used, while for storage, it's another discussion altogether. Similarly, there's a different philosophy in terms of deployment and so on. Spring Security is a framework that belongs to application-level security. In this section, you'll get a general picture of this security level and its implications.

Application-level security (figure 1.3) refers to everything that an application should do to protect the environment it executes in, as well as the data it processes and stores. Mind you, this isn't only about the data affected and used by the application. An application might contain vulnerabilities that allow a malicious individual to affect the entire system!

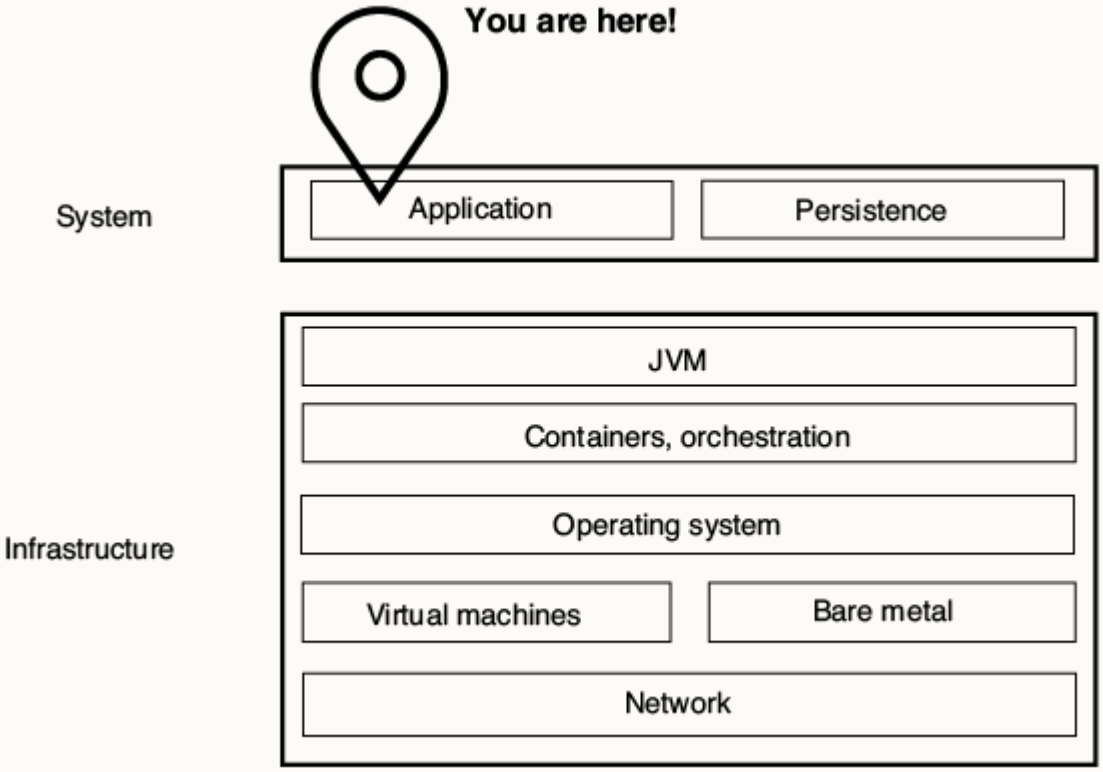


Figure 1.3 We implement security in a hierarchical manner, with each layer building on the foundation of the previous one. This book focuses on explaining how to use Spring Security, a framework that provides application-level security at the very top layer.

To be more explicit, let's discuss some practical cases. We'll consider a situation in which we deploy a system as in figure 1.4. This situation is common for a system designed using a microservices architecture, especially if you deploy it in multiple availability zones in the cloud.

NOTE If you're interested in implementing efficient cloud-oriented Spring apps, I strongly recommend *Cloud Native Spring in Action* by Thomas Vitale (Manning, 2022). In this book, the author focuses on all the needed aspects a professional has to know for implementing well-done Spring apps for cloud deployments.

With such services and microservices architectures, we can encounter various vulnerabilities, so you should exercise caution. As mentioned earlier, security is a cross-cutting concern that we design on multiple layers. When addressing the security concerns of one of the layers, the best practice is to assume as much as possible that the above layer doesn't exist. Think about the analogy with the castle in figure 1.2. If you manage the layer with 30 soldiers, you want to prepare them to be as strong as possible. And you do this despite knowing that before reaching them, someone would need to cross the fiery bridge.

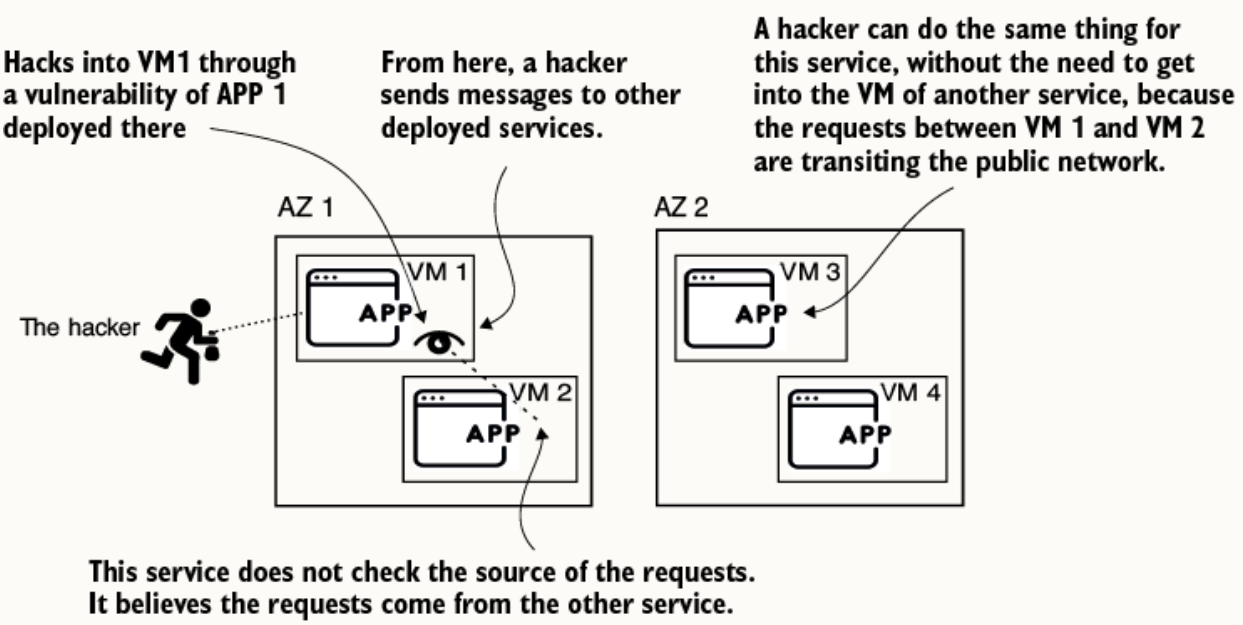


Figure 1.4 If a malicious user manages to get access to the virtual machine (VM) and there's no applied application-level security, a hacker can gain control of the other applications in the system. If communication occurs between two different availability zones (AZ), a malicious individual will find it easier to intercept the messages. This vulnerability allows them to steal data or to impersonate users.

With this in mind, let's consider that an individual driven by bad intentions would be able to log in to the virtual machine (VM) hosting the first application. Let's also assume that the second application doesn't validate the requests sent by the first application. The attacker can then exploit this vulnerability and control the second application by impersonating the first one.

Also, consider that we deploy the two services to two different locations. Then the attacker doesn't need to log in to one of the VMs as they can directly act in the middle of communications between the two applications.

NOTE An *availability zone* (AZ in figure 1.4) in terms of cloud deployment is a separate data center. This data center is situated far enough geographically (and has other dependencies) from other data centers of the same region that, if one availability zone fails, the probability that others are failing is minimal. In terms of security, an important aspect is that traffic between two different data centers generally needs special attention because it often goes across a public network.

I referred earlier to authentication and authorization. These are present in most applications. Through authentication, an application identifies a user (a person or another application). The purpose of identifying users is to be able to decide afterward what they should be allowed to do—that's authorization. I provide a lot of details on authentication and authorization, starting with chapter 3 and continuing throughout the book.

In an application, you often find the need to implement authorization in different scenarios. Consider another situation: most applications have restrictions regarding the user ac-

cessing certain functionality. Achieving this implies first the need to identify who creates an access to request a specific feature—that's authentication. We also need to know their privileges to allow the user to use that part of the system. As the system becomes more complex, you'll find different situations that require a specific implementation related to authentication and authorization.

For example, what if you'd like to authorize a particular component of the system against a subset of data or operations on behalf of the user? Let's say the printer needs access to read the user's documents. Should you simply share user credentials with the printer? But that allows the printer more rights than needed! And it also exposes the credentials of the user. Is there a proper way to do this without impersonating the user? These are essential questions and the kind of questions you encounter when developing applications: questions that we not only want to answer, but for which you'll see applications with Spring Security in this book.

Depending on the architecture chosen for the system, you'll find authentication and authorization at the level of the entire system, as well as for any of the components. And as you'll see further along in this book, with Spring Security, you'll sometimes prefer to use authorization even for different tiers of the same component. In chapter 11, we'll talk more about method security, which refers to this aspect. The design gets even more complicated when you have a predefined set of roles and authorities.

I would also like to bring to your attention data storage. Data at rest adds to the responsibility of the application. Your app shouldn't store all its data in a readable format. The application sometimes needs to keep the data either encrypted with a private key or hashed. Secrets such as credentials and private keys can also be considered data at rest. These should be carefully stored, usually in a secrets vault.

NOTE We classify data as "at rest" or "in transition." In this context, *data at rest refers* to data in computer storage or, in other words, persisted data. *Data in transition* applies to all the data that's exchanged from one point to another. Different security measures should, therefore, be enforced, depending on the data type.

Finally, an executing application must manage its internal memory as well. It may sound strange, but data stored in the heap of the application can also present vulnerabilities. Sometimes, the class design allows the app to store sensitive data, such as credentials or private keys, for a long time. In such cases, someone who has the privilege to make a heap dump could find these details and then use them maliciously.

With a short description of these cases, I hope I've managed to provide you with an overview of what we mean by application security, as well as to illustrate the complexity of this subject. Software security is a tangled subject. A person willing to become an expert in this field would need to understand (as well as to apply) and then test solutions for all the layers that collaborate within a system. In this book, however, we'll focus only on presenting all the details of what you specifically need to understand in terms of Spring Security. You'll find out where this framework applies and where it doesn't, how it helps, and why you should use it. Of course, we'll do this with practical examples that you should be able to adapt to your own unique use cases.

1.3 Why is security important?

The best way to start thinking about why security is important is from your point of view as a user. Like anyone else, you use applications, and these have access to your data. These can change your data, use it, or expose it. Think about all the apps you use, from your email to your online banking service accounts. How would you evaluate the sensitivity of the data that is managed by all these systems? How about the actions that you can perform using these systems? Similarly to data, some actions are more important than others. You don't care very much about some of those, while others are more significant. Maybe for you, it's not that important if someone can somehow manage to read some of your emails. But I bet you'd care if someone else could empty your bank accounts.

Once you've thought about security from your point of view, try to see a more objective picture. The same data or actions might have another degree of sensitivity to other people. Some people might care a lot more than you if their email is accessed and someone can read their messages. Your application should make sure to protect everything to the desired degree of access. Any leak that allows the use of data and functionalities, as well as the application, to affect other systems is considered a vulnerability, and you need to solve it.

Not considering enough security comes with a price that I'm sure you aren't willing to pay. In general, it's about money. But the cost can differ, and there are multiple ways through which you can lose profitability. It isn't only about losing money from a bank account or using a service without paying for it. These things indeed imply cost. The image of a brand or a company is also valuable, and losing a good image can be expensive—sometimes even more costly than the expenses directly resulting from the exploitation of a vulnerability in the system! The trust that users have in your application is one of its most valuable assets, and it can make the difference between success or failure.

Here are a few fictitious examples. Think about how you would see these as a user. How can these affect the organization responsible for the software?

- A back-office application should manage the internal data of an organization, but somehow, some information leaks out.
- Users of a ride-sharing application observe that money is debited from their accounts on behalf of trips that aren't theirs.
- After an update, users of a mobile banking application are presented with transactions that belong to other users.

In the first situation, the organization using the software, as well as its employees, can be affected. In some instances, the company could be liable and could lose a significant amount of money. In this situation, users don't have the choice to change the application, but the organization can decide to change the provider of their software.

In the second case, users will probably choose to change the service provider. The image of the company developing the application would be dramatically affected. The cost lost in terms of money in this case is much less than the cost in terms of image. Even if payments are returned to the affected users, the application will still lose some customers. This affects profitability and can even lead to bankruptcy. And in the third case, the bank could see dramatic consequences in terms of trust, as well as legal repercussions.

In most of these scenarios, investing in security is safer than what happens if someone exploits a vulnerability in your system. For all the examples, only a small weakness could cause each outcome. For the first example, it could be a broken authentication or a cross-site request forgery (CSRF). For the second and third examples, it could be a lack of method access control. And for all the examples, it could be a combination of vulnerabilities.

Of course, we can go even further from here and discuss the security in defense-related systems. If you consider money important, add human lives to the cost! Can you even imagine what could be the result if a healthcare system was affected? What about systems that control nuclear power? You can reduce any risk by investing early in the security of your application and by allocating enough time for security professionals to develop and test your security mechanisms.

NOTE The lesson learned from those who failed before you is that the cost of an attack is usually higher than the investment cost of avoiding the vulnerability.

In the rest of this book, you'll see examples of ways to apply Spring Security to avoid situations like the ones presented. I guess there will never be enough words written about

how important security is. When you have to make a compromise on the security of your system, try to estimate your risks correctly.

1.4 What will you learn in this book?

This book offers a practical approach to learning Spring Security. Throughout the rest of the book, we'll deep dive into Spring Security, proving concepts using simple to more complex examples. To get the most out of this book, you should be comfortable with Java programming, as well as with the basics of the Spring Framework. If you haven't used the Spring Framework or you don't feel comfortable yet using its basics, I recommend you first read *Spring Start Here*, another book I wrote (Manning, 2021). After reading that book, you can enhance your Spring knowledge with *Spring in Action*, *Sixth Edition*, by Craig Walls (Manning, 2022), as well as *Spring Boot: Up and Running* by Mark Heckler (O'Reilly Media, 2021).

In this book, you'll learn

- The architecture and basic components of Spring Security and how to use it to secure your application
- Authentication and authorization with Spring Security, including the OAuth 2 and OpenID Connect flows, and how these apply to a production-ready application
- How to implement security with Spring Security in different layers of your application
- Different configuration styles and best practices for using them in your project
- Using Spring Security for reactive applications
- Testing your security implementations

To make the learning process smooth for each described concept, we'll work on multiple simple examples.

When we finish, you'll know how to apply Spring Security for the most practical scenarios and understand where to use it and its best practices. I also strongly recommend that you work on all the examples that accompany the explanations.

Summary

- Spring Security is the leading choice for securing Spring applications. It offers a significant number of alternatives that apply to different styles and architectures.
- You should apply security in layers for your system, and for each layer, you need to use different practices.

- Security is a cross-cutting concern that you should consider from the beginning of a software project.
- Usually, the cost of an attack is higher than the cost of investment in avoiding vulnerabilities to begin with.
- Sometimes, the smallest mistakes can cause significant harm. For example, exposing sensitive data through logs or error messages is a common way to introduce vulnerabilities in your application.

Previous chapter
< Part 1. Say hello to Spring Security

Next chapter
2 Hello, Spring Security >